Onyx — AudioKit reference

Onyx · multi-process OS on Raspberry Pi 4

*The reference of **AudioKit** (`SD:/lib/audiokit.so`, `user/Kits/audiokit`): what it is for, how a program uses it, and every operation it exposes. The operations' part is made from the kit's headers by `tools/docgen/kitdocs.py` — the headers are the source. Overview of all the kits: The Kits.*

Contents

1. What it is
2. Using it
3. Index
4. `audiokit/audiokit.h`

What it is

AudioKit is sound: files played, notes on the General MIDI instruments, the FM synthesizer, mixing, WAV files written, and the system's output for a program that makes its own samples.

Include	`#include "audiokit/audiokit.h"`
Link	`lib/audiokit.imp.a`
Library	`SD:/lib/audiokit.so` — 236 entries in its table (`user/Kits/audiokit/audiokit.abi`, append-only)
Sources	`user/Kits/audiokit/`

Using it

Several programs play at once: the system mixes them, each with its own volume (Control Panel ▸ Sound).

A sound file (WAV, MP3, OGG, FLAC, MIDI, FM Tracker songs), played in the background:

```
#include "audiokit/audiokit.h"

if (ak_play ("SD:/music/song.mp3", 0) < 0) ax_putln (ak_play_error ());
ak_play_volume (80);                                  // 0..100
/* ... the program goes on ... */
ak_play_stop ();
```

Notes (the General MIDI instruments, a SoundFont):

```
ak_note_on (0, 60, 100);                              // channel, key (60 = middle C), velocity
kapi_msleep (400);
ak_note_off (0, 60);
```

A sound effect (the FM synthesizer's voices — what the games use):

```
ak_fm_start (0, 440000, 0, 200);                    // voice 0, 440 Hz (in milli-Hz),
volume 0..255
kapi_msleep (120);
ak_fm_stop (0);
```

The program's own samples (an emulator, a synthesizer): stereo, 16 bits, 44.1 kHz (`AUDIOKIT_RATE`).

```
if (ak_out_open (256, 1024) == 1)
{
    short frames[256 * 2];
    /* fill frames ... */
    ak_out_write (frames, 256);                     // waits for room
    ak_out_close ();
}
```

The microphone (the one chosen in Control Panel ▸ Sound: USB or I2S): mono, 16 bits, at the rate you ask. One program records at a time.

```
int rate = ak_in_open (16000);                      // 0: another program records, -1: no
microphone
if (rate > 0)
{
    short s[1600];
    int n = ak_in_read (s, 1600);                   // never waits: 0 = nothing yet, -1 =
gone
    int level = ak_in_peak (s, n);                  // 0..32767
    /* ... ak_wav_begin / ak_wav_write to keep it ... */
    ak_in_close ();
}
```

Index

Everything the headers declare, in their order — the details are in each header's part below.

Name	What it does	Header
`ak_info`	(a type)	`audiokit.h`
`ak_open`	The file opened (its kind from its extension, then its first bytes) -> 0	`audiokit.h`
`ak_read`	Up to `frames` frames at AUDIOKIT_RATE into out (2 shorts each) -> how many (0	`audiokit.h`
`ak_seek_ms`	1 done / 0	`audiokit.h`
`ak_info_of`	Up to `frames` frames at AUDIOKIT_RATE into out (2 shorts each) -> how many (0	`audiokit.h`
`ak_close`	Up to `frames` frames at AUDIOKIT_RATE into out (2 shorts each) -> how	`audiokit.h`

Name	What it does	Header
	many (0	
`ak_tags`	What a sound file says about itself (MP3	`audiokit.h`
`ak_tags_read`	1 / 0: not a sound file of ours, unreadable	`audiokit.h`
`ak_play`	0 started / -1 (ak_play_error says why)	`audiokit.h`
`ak_play_stop`	the file stopped and closed -> AK_STOPPED (no file: nothing)	`audiokit.h`
`ak_play_pause`	1: paused, 0: playing again (no file: nothing)	`audiokit.h`
`ak_play_state`	AK_* (the file's; the live notes aside)	`audiokit.h`
`ak_play_pos_ms`	where the file is heard, ms from its start (0: no file)	`audiokit.h`
`ak_play_len_ms`	the file's length, ms (0: no file, not known)	`audiokit.h`
`ak_play_seek_ms`	the file goes on from there -> 1 done / 0 (no file, it cannot seek)	`audiokit.h`
`ak_play_volume`	0..100 (-1: only ask) -> the volume	`audiokit.h`
`ak_play_error`	the last error's words ("": none)	`audiokit.h`
`ak_play_wait`	waits while a file plays, ms at most (-1: no limit) -> its state	`audiokit.h`
`ak_play_keep_output`	on	`audiokit.h`
`ak_note_on`	0 / -1 (no SoundFont: ak_play_error)	`audiokit.h`
`ak_note_off`	Live notes on the General MIDI synthesizer (the default SoundFont), 16 channels (9	`audiokit.h`
`ak_program`	the channel's instrument, 0..127 (General MIDI)	`audiokit.h`
`ak_control`	a MIDI controller (7 volume, 10 pan, 64 pedal...)	`audiokit.h`
`ak_pitch_bend`	-8192..8191	`audiokit.h`

Name	What it does	Header
`ak_notes_off`	everything silent now	`audiokit.h`
`ak_out_open`	kapi_sound_config's -> 1 ours, 0 busy, -1 no sound	`audiokit.h`
`ak_out_write`	all of them, waiting for room -> n (< n: closed)	`audiokit.h`
`ak_out_free`	frames that would be taken without waiting	`audiokit.h`
`ak_out_queued`	frames written and not yet played	`audiokit.h`
`ak_out_close`	(Not together with the player	`audiokit.h`
`ak_in_open`	-> rate (0 asked: 16000), 0 another program records, -1 no microphone	`audiokit.h`
`ak_in_read`	never waits -> the samples taken (0: none yet), -1 closed / the device gone	`audiokit.h`
`ak_in_close`	The one the Sound applet chose (a USB microphone or a headset's, an I2S microphone on the header	`audiokit.h`
`ak_in_peak`	the loudest sample, 0..32767 (a level meter)	`audiokit.h`
`ak_gain_s16`	buf *= gain, saturated, in place	`audiokit.h`
`ak_mix_s16`	dst += src * gain, saturated	`audiokit.h`
`ak_mono_to_stereo`	the first `frames` shorts spread, in place	`audiokit.h`
`ak_volume_gain`	0..100 on the ear's curve -> a 16.16 gain	`audiokit.h`
`ak_resampler_new`	the rates in Hz -> 0: a rate <= 0	`audiokit.h`
`ak_resample`	in_frames frames in -> the frames written to out (out_cap at most)	`audiokit.h`
`ak_resampler_free`	in_frames frames in -> the frames written to out (out_cap at most)	`audiokit.h`
`ak_f32_to_s16`	(FPU programs) Two float	`audiokit.h`

Name	What it does	Header
	channels (-1..1) -> 16-bit stereo, through the soft limiter (Koton's	
`ak_soft_clip`	(FPU programs) Two float channels (-1..1) -> 16-bit stereo, through the soft limiter (Koton's	`audiokit.h`
`ak_note_mhz`	a MIDI key's frequency, milli-Hz (69 = A4 = 440000)	`audiokit.h`
`ak_note_key`	a note (0 = C .. 11 = B) and an octave -> the key (C4 = 60)	`audiokit.h`
`ak_note_octave_mhz`	... its frequency, milli-Hz	`audiokit.h`
`ak_note_name`	"C4", "F#3" (60 = C4)	`audiokit.h`
`ak_note_parse`	"C4", "f#3", "Bb2" -> the key, -1	`audiokit.h`
`ak_wav_header`	The 44-byte header of a 16-bit PCM file of data_bytes bytes of samples -> 44.	`audiokit.h`
`ak_wav_save`	stereo -> 0 / -1	`audiokit.h`
`ak_wav_begin`	0: not created	`audiokit.h`
`ak_wav_write`	-> the frames taken	`audiokit.h`
`ak_wav_end`	0 / -1	`audiokit.h`
`ak_soundfont_default`	The default SoundFont	`audiokit.h`
`ak_soundfont_name`	its name ("": none loaded)	`audiokit.h`
`ak_soundfont_find`	A SoundFont's file	`audiokit.h`
`ak_soundfont_load`	-> an ms::SoundFont * of the caller's (0: err)	`audiokit.h`
`ak_soundfont_free`	A SoundFont's file	`audiokit.h`
`ak_soundfont_prefer`	The file the default SoundFont is to be (a program's setting	`audiokit.h`
`ak_synth_new`	on the default SoundFont, at AUDIOKIT_RATE -> 0: none	`audiokit.h`
`ak_synth_free`	the synthesizer freed (the default SoundFont stays loaded)	`audiokit.h`
`ak_synth_midi`	0x90 note on, 0x80 off,	`audiokit.h`

Name	What it does	Header
	0xC0 program...	
`ak_synth_render`	never allocates, never calls the kernel	`audiokit.h`
`ak_fm_instrument`	0 / -1	`audiokit.h`
`ak_fm_start`	voice 0..15, volume 0..255	`audiokit.h`
`ak_fm_stop`	-1: all (the releases are heard)	`audiokit.h`
`ak_fm_silence`	all silent at once	`audiokit.h`
`ak_fm_render`	the voices' next frames written to out (off line: an export)	`audiokit.h`
`ak_fm_live`	0: the player leaves the voices alone -- the program renders them (ak_fm_render); 1 (at first): heard	`audiokit.h`
`ak_reverb_new`	for samples at that rate (Hz) -> 0: no memory	`audiokit.h`
`ak_reverb_set`	< 0: unchanged	`audiokit.h`
`ak_reverb_process`	mono in, stereo out	`audiokit.h`
`ak_reverb_mute`	its tail dropped: the delay lines cleared	`audiokit.h`
`ak_reverb_free`	the reverb and its buffers freed (0: nothing)	`audiokit.h`
`ak_chorus_new`	(MeltySynth's own: 0.002, 0.0019, 0.4)	`audiokit.h`
`ak_chorus_process`	stereo in, the chorused (delayed) signal alone out	`audiokit.h`
`ak_chorus_mute`	what it holds dropped: the delay lines cleared	`audiokit.h`
`ak_chorus_free`	the chorus and its buffers freed (0: nothing)	`audiokit.h`

`audiokit/audiokit.h`

audiokit.h -- AudioKit, Onyx's shared sound library (SD:/lib/audiokit.so; docs/03 "AudioKit").

Everything about sound that more than one program can use, in one copy for the whole system:

```
  files      a sound file of any kind -- MP3, FLAC, WAV, Ogg Vorbis, MIDI (through a
SoundFont) --
             read as 16-bit stereo frames at the output's rate (ak_open, ak_read, ak_seek_ms)
  the player a file played in the background on the system's output, and notes played live on
```

```
a
             General MIDI synthesizer, mixed (ak_play, ak_note_on ...): one line to make a
sound
  the output the system's sound output for a program that makes its own frames (ak_out_*)
  the input  the microphone chosen in the Sound applet, USB or I2S (ak_in_*)
  mixing     gain, mix with saturation, mono to stereo, float to 16 bits with a soft limiter,
             a rate converter (ak_mix_s16, ak_f32_to_s16, ak_resample_*)
  notes      a note's frequency, its name, a name's note (ak_note_*)
  WAV        a WAV file written (ak_wav_header, ak_wav_save)
  the synthesizer  MeltySynth, the SoundFont synthesizer of Koton: its C++ interface
             (Apps/koton/synth/meltysynth.h: ms::Synthesizer, ms::soundfont_load, the reverb
and
             the chorus) is exported as it is; ak_synth_* is the same for C and for BASIC
  the decoders     minimp3, dr_flac, dr_wav and stb_vorbis: their own C interfaces
             (Apps/media/codecs.h) are exported as they are, for a program that wants them
raw
```

A program links lib/audiokit.imp.a (user/Makefile) and calls these as plain functions: the library is opened before main (user/Runtime/lib.h). The interface is append-only (audiokit/audiokit.abi). Frames are always interleaved 16-bit stereo (left, right) unless said; "the rate" is AUDIOKIT_RATE, the system output's (44100 Hz).

The entries that take or return float / double are for programs built with the FPU (every newlib app, /bin/basic, the games built with CXXFLAGS_FP): an integer-only program (-mgeneral-regs-only) uses the others -- all the file, player, output, note and 16-bit mixing calls are integer.

MIT License

Copyright (c) 2026 Stéphane Wegener and the Onyx contributors

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE. (The synthesizer is MeltySynth, MIT, Nobuaki Tanaka; minimp3 CC0; dr_flac / dr_wav and stb_vorbis public domain / MIT: docs/LICENSING.md.)

```
#define AUDIOKIT_RATE   44100           // the frames of every call: 16-bit stereo at this
rate
```

a sound file

```
struct ak_info
{
    int rate;               // the file's own rate (ak_read gives AUDIOKIT_RATE)
    int channels;               // what the file has (ak_read gives stereo)
```

```
    int bits;               // its sample size (0: not known)
    int kbps;               // its bit rate (0: not known)
    long long length_ms;            // its length (0: not known)
    char format[8];             // "MP3", "FLAC", "WAV", "OGG", "MIDI", "FMS"
};
typedef struct ak_stream ak_stream;
```

The file opened (its kind from its extension, then its first bytes) -> 0: err (cap bytes) says why. A MIDI file plays through the default SoundFont (ak_soundfont_default); an FM Song (.fms, FM Tracker's) on the FM synthesizer (one read at a time per process).

```
ak_stream *ak_open (const char *path, char *err, int cap);
```

Up to `frames` frames at AUDIOKIT_RATE into out (2 shorts each) -> how many (0: the end).

```
int ak_read (ak_stream *s, short *out, int frames);
int ak_seek_ms (ak_stream *s, long long ms);        // 1 done / 0
void ak_info_of (ak_stream *s, struct ak_info *out);
void ak_close (ak_stream *s);
```

What a sound file says about itself (MP3: ID3v2 / ID3v1; FLAC and Ogg: Vorbis comments; WAV: LIST INFO; MIDI: the first track's name; an FM Song: its title and author) -- read only, nothing decoded. What is missing comes from the path: the file's name (a leading "03 - ": the track), its folder (the album), the folder above (the artist).

```
struct ak_tags
{
    char title[128], artist[96], album_artist[96], album[128], genre[48];
    int year, track, disc;          // 0: not known
    int duration_ms;            // 0: not known
    char format[8];             // "MP3", "OGG", "FLAC", "WAV", "MIDI", "FMS"
    long long cover_offset;         // a picture inside the file (JPEG / PNG bytes): where,
and
    unsigned cover_length;          // how long (0: none)
    int reserved[8];
};
int ak_tags_read (const char *path, struct ak_tags *out);   // 1 / 0: not a sound file of
ours, unreadable
```

the player: a file in the background, notes played live

One per process: a thread of its own that holds the system's output while it has something to play (another program playing: AK_BUSY until it lets go). The file and the live notes are mixed.

```
#define AK_STOPPED  0
#define AK_PLAYING  1
#define AK_PAUSED   2
#define AK_BUSY     3           // playing, but another program holds the output

int ak_play (const char *path, int loop);       // 0 started / -1 (ak_play_error says why)
void ak_play_stop (void);   // the file stopped and closed -> AK_STOPPED (no file: nothing)
void ak_play_pause (int on);    // 1: paused, 0: playing again (no file: nothing)
int ak_play_state (void);               // AK_* (the file's; the live notes aside)
long long ak_play_pos_ms (void);    // where the file is heard, ms from its start (0: no
file)
long long ak_play_len_ms (void);    // the file's length, ms (0: no file, not known)
int ak_play_seek_ms (long long ms); // the file goes on from there -> 1 done / 0 (no file, it
cannot seek)
```

```
int ak_play_volume (int volume);            // 0..100 (-1: only ask) -> the volume
const char *ak_play_error (void);           // the last error's words ("": none)
int ak_play_wait (int ms);              // waits while a file plays, ms at most (-1: no
limit) -> its state
```

on: the player keeps the system's output once it has it, also when it has nothing to play (a program that plays the kernel's voices as well: BASIC's SOUND / PLAY) -- else it lets go after ~0.6 s of silence, so that other programs can play.

```
void ak_play_keep_output (int on);
```

Live notes on the General MIDI synthesizer (the default SoundFont), 16 channels (9: the drums).

```
int ak_note_on (int channel, int key, int velocity);    // 0 / -1 (no SoundFont:
ak_play_error)
void ak_note_off (int channel, int key);
void ak_program (int channel, int program);     // the channel's instrument, 0..127 (General
MIDI)
void ak_control (int channel, int controller, int value);   // a MIDI controller (7 volume,
10 pan, 64 pedal...)
void ak_pitch_bend (int channel, int value);        // -8192..8191
void ak_notes_off (void);               // everything silent now
```

the system's output, for a program that makes its own frames

(Not together with the player: both want the output.)

```
int ak_out_open (int chunk_frames, int ahead);      // kapi_sound_config's -> 1 ours, 0 busy,
-1 no sound
int ak_out_write (const short *frames, int n);      // all of them, waiting for room -> n (<
n: closed)
int ak_out_free (void);                 // frames that would be taken without waiting
int ak_out_queued (void);               // frames written and not yet played
void ak_out_close (void);
```

the system's input: a microphone (kapi v99)

The one the Sound applet chose (a USB microphone or a headset's, an I2S microphone on the header: SD:/etc/sound.ini "input ="), one program at a time; the samples 16-bit MONO at the rate asked (converted from the device's), the DC taken away and the input's gain applied by the kernel.

```
int ak_in_open (int rate);              // -> rate (0 asked: 16000), 0 another program
records, -1 no microphone
int ak_in_read (short *out, int max);           // never waits -> the samples taken (0: none
yet), -1 closed / the device gone
void ak_in_close (void);
int ak_in_peak (const short *samples, int n);       // the loudest sample, 0..32767 (a level
meter)
```

mixing and conversion (16-bit stereo frames; gains in 16.16: 65536 = 1)

```
void ak_gain_s16 (short *buf, int frames, int gain);    // buf *= gain, saturated, in place
void ak_mix_s16 (short *dst, const short *src, int frames, int gain);   // dst += src * gain,
saturated
void ak_mono_to_stereo (short *buf, int frames);            // the first `frames` shorts
spread, in place
int ak_volume_gain (int volume);            // 0..100 on the ear's curve -> a 16.16 gain

typedef struct ak_resampler ak_resampler;       // a rate converter (linear), stereo
```

```
ak_resampler *ak_resampler_new (int in_rate, int out_rate); // the rates in Hz -> 0: a rate <= 0
```

in_frames frames in -> the frames written to out (out_cap at most); *used: the input frames taken.

```
int ak_resample (ak_resampler *r, const short *in, int in_frames, short *out, int out_cap, int *used);
void ak_resampler_free (ak_resampler *r);
```

(FPU programs) Two float channels (-1..1) -> 16-bit stereo, through the soft limiter (Koton's: straight up to 0.75, then a knee that never passes 1) instead of clipping.

```
void ak_f32_to_s16 (const float *left, const float *right, short *out, int frames, float gain);
float ak_soft_clip (float x);
```

notes

```
int ak_note_mhz (int key);              // a MIDI key's frequency, milli-Hz (69 = A4 = 440000)
int ak_note_key (int note, int octave);         // a note (0 = C .. 11 = B) and an octave -> the key (C4 = 60)
int ak_note_octave_mhz (int note, int octave);      // ... its frequency, milli-Hz
void ak_note_name (int key, char *out8);        // "C4", "F#3" (60 = C4)
int ak_note_parse (const char *name);           // "C4", "f#3", "Bb2" -> the key, -1
```

WAV

The 44-byte header of a 16-bit PCM file of data_bytes bytes of samples -> 44.

```
int ak_wav_header (unsigned char *out44, int rate, int channels, unsigned data_bytes);
int ak_wav_save (const char *path, const short *frames, int n, int rate);   // stereo -> 0 / -1
```

A long file written as it is made (an export): its length first, then its frames, then the end.

```
typedef struct ak_wav ak_wav;
ak_wav *ak_wav_begin (const char *path, int rate, int channels, long long frames);  // 0: not created
int ak_wav_write (ak_wav *w, const short *frames, int n);   // -> the frames taken
int ak_wav_end (ak_wav *w);                 // 0 / -1
```

the synthesizer, for C and BASIC (C++: meltysynth.h's ms::Synthesizer is exported too)

The default SoundFont: the first .sf2 of SD:/res/soundfonts (the package GeneralUser GS), loaded once per process -> an ms::SoundFont *, 0: err says why.

```
void *ak_soundfont_default (char *err, int cap);
const char *ak_soundfont_name (void);           // its name ("": none loaded)
```

A SoundFont's file: `preferred` if it exists, else the first .sf2 of SD:/res/soundfonts, SD:/koton/soundfonts, SD:/music/soundfonts, SD:/music, SD:/apps/koton.app -> 1 (out: its path) / 0.

```
int ak_soundfont_find (const char *preferred, char *out, int cap);
void *ak_soundfont_load (const char *path, char *err, int cap); // -> an ms::SoundFont * of the caller's (0: err)
void ak_soundfont_free (void *sf);
```

The file the default SoundFont is to be (a program's setting; before the first MIDI sound). "": none.

```
void ak_soundfont_prefer (const char *path);
typedef struct ak_synth ak_synth;
ak_synth *ak_synth_new (void);              // on the default SoundFont, at AUDIOKIT_RATE -> 0: none
void ak_synth_free (ak_synth *s);   // the synthesizer freed (the default SoundFont stays loaded)
void ak_synth_midi (ak_synth *s, int channel, int command, int data1, int data2);   // 0x90 note on, 0x80 off, 0xC0 program...
void ak_synth_render (ak_synth *s, short *out, int frames); // never allocates, never calls the kernel
```

the FM synthesizer: the system's voices (fmsynth.h; they were the kernel's until 2026-10-05)

16 voices, each a plain wave or a two-operator FM instrument, played by the player's thread, mixed with the file and the MIDI notes. `wave` and the instrument are the kapi's names (SOUND_SQUARE ... SOUND_FM, struct kapi_fm_instrument). A note sounds until it is stopped.

```
struct kapi_fm_instrument;
int ak_fm_instrument (int voice, const struct kapi_fm_instrument *ins); // 0 / -1
int ak_fm_start (int voice, unsigned milli_hz, int wave, int volume);       // voice 0..15, volume 0..255
void ak_fm_stop (int voice);                            // -1: all (the releases are heard)
void ak_fm_silence (void);                          // all silent at once
void ak_fm_render (short *out, int frames); // the voices' next frames written to out (off line: an export)
void ak_fm_live (int on);   // 0: the player leaves the voices alone -- the program renders them (ak_fm_render); 1 (at first): heard
```

effects (FPU programs): MeltySynth's reverb (Freeverb) and chorus

```
typedef struct ak_reverb ak_reverb;
ak_reverb *ak_reverb_new (int rate);    // for samples at that rate (Hz) -> 0: no memory
void ak_reverb_set (ak_reverb *r, float room, float damp, float wet, float width);  // < 0: unchanged
void ak_reverb_process (ak_reverb *r, const float *in, float *left, float *right, int frames);  // mono in, stereo out
void ak_reverb_mute (ak_reverb *r); // its tail dropped: the delay lines cleared
void ak_reverb_free (ak_reverb *r); // the reverb and its buffers freed (0: nothing)
typedef struct ak_chorus ak_chorus;
ak_chorus *ak_chorus_new (int rate, float delay_s, float depth_s, float hz);    // (MeltySynth's own: 0.002, 0.0019, 0.4)
void ak_chorus_process (ak_chorus *c, const float *inL, const float *inR, float *outL, float *outR, int frames);    // stereo in, the chorused (delayed) signal alone out
void ak_chorus_mute (ak_chorus *c); // what it holds dropped: the delay lines cleared
void ak_chorus_free (ak_chorus *c); // the chorus and its buffers freed (0: nothing)

}
#endif
```